

——— Review questions 1

R1.a (4 Points) Describe the slotted Aloha protocol. Does the protocol provide adaptation to congestion? Explain your answer

* Assumption/Set up

- All frames consist of exactly L bits.
- Time is divided into slots of size L/R seconds (a slot equals the time to transmit one frame).
- Nodes start to transmit frames only at the beginnings of slots.
- The nodes are synchronized so that each node knows when the slots begin.
- If two or more frames collide in a slot, then all the nodes detect the collision event before the slot ends.

* Operation

- When the node has a fresh frame to send, it waits until the beginning of the next slot and transmits the entire frame in the slot.
- If there isn't a collision, the node can prepare a new frame for transmission, if it has one.
- If there is a collision, the node detects the collision before the end of the slot. The node retransmits its frame in each subsequent slot with probability p until the frame is transmitted without a collision.

Adaptation to congestion is not provided. Slotted Aloha only makes sure one frame is successfully transmitted or keeps trying. Collision does not equals congestion.

R1.b (5 Points) Describe multiplexing and demultiplexing of TCP and UDP protocols.

UDP

- Unique application process for each active source port number

Source side multiplexing

- Application process passes data, source port number, destination port number to transport layer
- Transport layer creates segments that includes application data, puts source & destination port numbers in packet header as well as other such as length and checksum, and passes destination IP address to network layer (UDP header is light = 8 bytes)
- Network layer encapsulates the segment into IP datagram and send it using best-effort service

Destination side demultiplexing

- Obtain destination port number from UDP header
- Pass to socket = (destination IP address, destination port): i.e., to application process associated with destination port

TCP

- Unique application process for each active combination of (source port number, destination port number, destination IP address)

Source side multiplexing

- Application process passes data, source port number, destination port number to transport layer
- Transport layer puts source & destination port numbers in packet header including sequence number, acknowledgement number and others fields (control bits, check sum, etc...) and passes destination IP address to network layer (TCP header is 20 bytes)
- Network layer encapsulates the segment into IP datagram and send it using best-effort service

Destination side demultiplexing

- Obtain source and destination port numbers from TCP header
- Obtain sequence number from TCP header
- Request dropped packets (by sending duplicate ACK with sequence number of expected bytes)
- Put packets in order
- Pass to socket = (src IP, src port, dest IP, dest port): i.e., to application process associated with (src IP, src port, dest port)

R1.c (5 Points) Discuss why we need hierarchical routing in the internet. (Very) Briefly describe intra-AS and inter-AS routing.

Hierarchical routing:

Scale/Complexity. As the number of routers becomes large, the overhead involved in computing, storing, and communicating routing information becomes prohibitive. Given hundreds of millions of hosts in the internet currently, storing routing information at each of these hosts would clearly require enormous amounts of memory. The overhead required to broadcast LS updates among all of the routers in the public Internet would leave no bandwidth left for sending data packets! A distance-vector algorithm that iterated among such a large number of routers would surely never converge.

Administrative autonomy. A company wants to run its routers as it pleases: freely choose a routing algorithm or hide aspects of its network's internal organization from the outside due to some security issues. Ideally, an organization should be able to run and administer its network as it wishes, while still being able to connect its network to other outside networks.

=> Thus we need hierarchical routing

Intra-AS routing

Routers within the same AS all **run the same routing algorithm** (e.g., LS or DV) and **have information about each other.** Each of these routers knows how to **forward packets along the optimal path to any destination within AS.**

It will be necessary, of course, to **connect ASs to each other**, and thus **one or more of the routers in an AS** will have the added task of being responsible for **forwarding packets to destinations outside the AS**; these routers are called **gateway routers**.

Inter-AS routing:

Handles two tasks: **obtaining reachability information from neighboring ASs** and **propagating the reachability information to all routers internal to the AS.** Since the inter-AS routing protocol involves communication between two ASs, the **two communicating ASs must run the same inter-AS routing protocol.**(BGP)

The problem of scale is solved because an intra-AS router need only know about routers within its AS. The problem of administrative authority is solved since an organization can run whatever intra-AS routing protocol (RIP, OSPF) it chooses; however, each pair of connected ASs needs to run the same inter-AS routing protocol to exchange reachability information.

R1.d (6 Points) Compare and contrast Link State and Distance Vector algorithm.

They both are intra-AS algorithms that populate the forwarding table (**calculate the least-cost path**) of routers inside an AS.

- Nature of the algorithm:

LS: A **global routing** algorithm using **complete, global knowledge** about the network (**connectivity and link costs**). The algorithm has to somehow obtain this information **before** actually performing the calculation. The calculation itself can be run at **one site** (a centralized global routing algorithm) or **replicated at multiple sites**.

DV: A **decentralized** routing algorithm, the calculation of the **least-cost path** is carried out in an **iterative, distributed** manner. No node has complete information about the costs of all network links. Instead, each node begins with only the knowledge of the costs of its **own directly attached links**. Then, through an iterative process of calculation and exchange of information with its neighboring nodes, a node gradually calculates the least-cost path to a destination or set of destinations.

- Who to talk to

In LS, each node talks with **all other nodes** (via broadcast)

In DV, each node talks to only its **directly connected neighbors**

- Message content

In LS, each node tells the **costs** of its directly connected links.

In DV, each node provides its neighbors with **least-cost estimates from itself to all the nodes** (that it knows about) in the network.

- Message complexity.

LS requires each node to know **the cost of each link** in the network. This requires $O(|N| |E|)$ messages to be sent. Whenever a link cost **changes**, the new link cost must be **sent to all** nodes.

DV algorithm requires message exchanges between directly connected neighbors at each iteration. The time needed for the algorithm to converge can depend on many factors. When link costs change, the DV algorithm will propagate the results of the changed link cost only if the new link cost results in a changed least-cost path for one of the nodes attached to that link.

- Speed of **convergence**.

LS is an $O(|N|^2)$ algorithm requiring $O(|N| |E|)$ messages.

The DV algorithm can converge **slowly** and can have routing loops while the algorithm is converging. DV also suffers from the **count-to-infinity problem**.

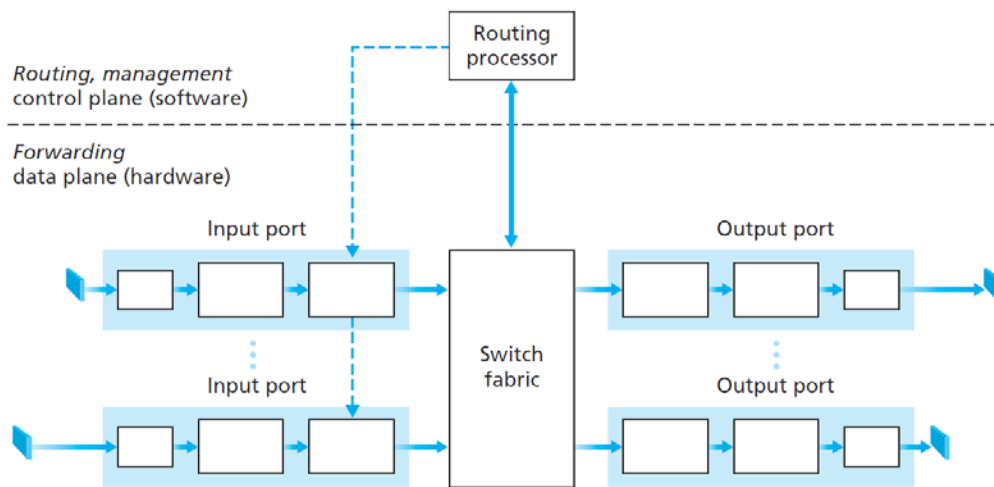
- **Robustness**. What if a router fails, misbehaves, or is sabotaged?

Under LS, a router could broadcast an incorrect cost for one of its attached links (but no others). A node could also corrupt or drop any packets it received as part of an LS broadcast. But an LS node is computing only its own **forwarding tables**; other nodes are performing similar calculations for themselves. This means route calculations are somewhat **separated** under LS, providing a degree of robustness.

Under DV, a node can advertise incorrect least-cost paths to any or all destinations. At each iteration, a node's calculation in DV is passed on to its neighbor and then indirectly to its neighbor's neighbor on the next iteration. In this sense, an incorrect node calculation can be diffused through the entire network under DV.

Review questions 2

R2.a (7Points) List and briefly describe the **architecture of routers** and **list the types of switching fabric**. Do the input and output ports implement protocols? What layer do they refer to?



Four router components can be identified:

- **Input ports.** An input port performs several key functions. It performs the **physical layer** function of **terminating an incoming physical link** at a router; An input port also performs **link-layer** functions needed to **interoperate** with the link layer **at the other side** of the incoming link; Most crucially, the **lookup function** is also performed at the input port: the **forwarding table** is consulted to **determine the router output port** to which an arriving packet will be forwarded via the switching fabric. **C**ontrol packets (packets carrying routing protocol information) are forwarded to the routing processor.

- **Switching fabric.** The switching fabric **connects the router's input ports to its output ports**. This switching fabric is completely contained **within the router**!

Switching via **memory**: switching between input and output ports being done under direct control of the **CPU** (routing processor). Input and output ports functioned as traditional I/O devices in a traditional operating system.

Switching via a **bus**: An input port transfers a packet directly to the output port over a **shared bus**, without intervention by the routing processor. The packet is received by all output ports, but only the port that **matches the label** will keep the packet.

Switching via **crossbar**: to overcome the bandwidth limitation of a single, shared bus, A crossbar switch is an **interconnection network** consisting of **2N buses** that connect N input ports to N output ports. Crossbar networks are capable of **forwarding multiple packets in parallel**.

- **Output ports.** An output port **stores packets** received from the switching fabric and transmits these packets on the outgoing link by performing the necessary link-layer and physical-layer functions. This includes **selecting and de-queueing** packets for transmission.

- **Routing processor.** The routing processor executes the **routing protocols**, maintains **routing tables** and attached **link state information**, and computes the **forwarding table** for the router.

Input and output ports implemented protocols referring to **link-layer and physical layer**

R2.b (7Points) List the main examples of "taking turns" and "Random access" Medium Access Control (MAC) protocols and briefly describe them. Discuss their advantages/disadvantages. In what traffic regimes do they best perform?

Taking turns:

- Channel partitioning

- TDMA, FDMA: partition the channel based on time and frequency

- (+) no collisions, perfectly fair,

- (-) limited to average rate of allocated resources even only 1, must always wait for turns

- (-) unused slots go idle, unused transmission time in frequency bands go idle

- CDMA: partition the channel based on code

- (+) different node can transmit simultaneously the receiver receive correctly, codes can be allocated to multiple access channel uses

- Round robin

- Polling: Needs a master node. The master node polls each of the nodes in a round-robin fashion. Each node being polled can transmit up to some maximum number of frames. The master node polls each of the nodes in a cyclic manner. (+) eliminates the collisions and empty slots that plague random access protocols. This allows polling to achieve a much higher efficiency.

- Token-passing: No master, a small, special-purpose frame known as a token is exchanged among the nodes in some fixed order. (+) If you want to transmit get a token. No collisions! Decentralized and highly efficient.

- (-) Latency: a polling delay—the amount of time required to notify a node that it can transmit.

- (-) Overhead

- (-) Single point of failure: In polling, if the master node fails, the entire channel becomes inoperative. In token passing, the failure of one node can crash the entire channel. Or even if a node accidentally neglects to release the token

Random access:

- (+) No turns! thus (-) collisions

- (+) When node has packet to send: transmit at full channel data rate R , no a priori coordination among nodes

- (-) Need protocol for telling when it's your turn

- (-) And for dealing with collisions - specifies: –how to detect collisions –how to recover from collisions

Examples:

- slotted ALOHA (R1a) ==> (-) synchronization between all nodes (-) wait for slot

- ALOHA: (+) transmit immediately not waiting for slots -> (-) collision probability increases. However it is

- (+) simpler, (+) no synchronization

- (-) poor efficiency: slotted ALOHA: 37%. ALOHA: 18%

- CSMA, CSMA/CD, CSMA/CA : if channel sensed idle: transmit entire frame, if channel sensed busy, defer transmission (-) cost for sensing the channel

- CD: collisions detected within short time

- (+) colliding transmissions aborted, (+) reducing channel wastage

Best-suited traffic regimes:

Taking turns protocols (and especially channel partition) are better under heavy traffic conditions (nodes have a lot to transmit). In fact, we don't waste time assigning unused resource to nodes with no traffic. Under heavy traffic, random protocols may incur a lot of collisions and errors.

With light arrival rates random protocols are better because we assign traffic more frequently to nodes that have traffic to be delivered (and we don't waste resource assigning it to nodes that are not going to use it - channel partition, and we don't have fixed overhead - taking turns). Random protocols are also better in terms of delay because resource allocation is more flexible.

R2.c (8Points) Provide a brief overview of **TCP congestion control**. Why the **timeout event** and the **duplicate acknowledgment events** generate a different reaction of the protocol?

The approach taken by TCP is to have each sender limit the **rate** at which it sends traffic into its connection as a function of perceived network congestion. Given **ACKs** indicating a **congestion-free source-to-destination path** and **loss events** indicating a **congested path**, TCP's strategy for adjusting its transmission rate is to **increase its rate in response to arriving ACKs until a loss event occurs**, at which point, the transmission rate is decreased. The TCP sender thus increases its transmission rate to probe for the rate that at which congestion onset begins, backs off from that rate, and then to begins probing again to see if the congestion onset rate has changed.

TCP keeps track of an additional variable, the congestion window. The **congestion window** (cwnd) imposes a constraint on the **rate** at which a TCP sender can send traffic in a connection. The constraint permits the sender to send cwnd bytes of data into the connection; at the end of the RTT the sender receives acknowledgement for the data. Thus the sender's **send rate** is roughly **cwnd/RTT bytes/sec**.

TCP will take the arrival of the ACKs as an indication that all is well—that segments being transmitted into the network are being successfully delivered to the destination—and will use ACKs to increase its congestion window size (and hence its transmission rate) Note that if acknowledgments arrive at a relatively slow rate (e.g., if the end-end path has high delay or contains a low-bandwidth link), then the congestion window will be increased at a relatively slow rate. On the other hand, if acknowledgments arrive at a high rate, then the congestion window will be increased more quickly. (TCP ACKs are cumulative)

Let us define a **"loss event"** at a TCP sender as the occurrence of either a **timeout** or the **receipt of three duplicate ACKs from the receiver**. A loss event is taken by the sender to be an indication of congestion on the sender-to-receiver path. the TCP sender's rate should be decreased when a segment is lost

Reaction of the protocol is different in "loss event"

- Timeout: **retransmit** not-yet-acked segment with **smallest sequence number**
- 3 duplicated ACKs: perform fast retransmit: retransmitting the missing segment **before that segment's timer** expires since this indicates the segment following the segment that has been ACKed three times has been lost. This is because (fortunately) the sender **can often detect** packet loss well before the timeout event occurs. Since timeout is relatively **long** -> **increase end-to-end delay** if you wait for timeout.

* Alternatively, you can describe in term of the states and their transitions to illustrate the principles behind. Slow Start - Congestion Avoidance - Fast Recovery (which is recommended, not required by TCP)

Increase Sending Rate Phase Options:

1. When cwnd is below threshold, sender in slow-start phase, window **starts slow but grows exponentially**.
2. When cwnd is above threshold, sender is in congestion-avoidance phase, **window grows linearly**.

Decrease Sending Rate Phase Options:

1. When a triple duplicate ACK occurs, threshold is set to cwnd/2 and cwnd is set to threshold.
2. When timeout occurs, threshold is set to cwnd/2 and cwnd is set to 1 MSS.

THE REASON WHY THE PROTOCOL REACTS DIFFERENTLY IN CASE OF TIMEOUT AND 3 DUPLICATED ACKS:

3 dup ACKs indicates network is still **capable of delivering some segments**

Timeout indicates a **"more alarming"** congestion scenario: connection is **broken in both directions**

*Note TCP protocol reacts **only at 3 dup** ACKs, not just 1 dup ACK: It is a trade-off: **waiting for more packets** (rather than just 1) to **avoid retransmitting prematurely** in the face of packet reordering. (of course there are situations that this policy will slow things down)

R2.d (8Points) Describe **uncontrolled flooding**, **controlled flooding** and **spanning tree** broadcast algorithms. Rank these techniques according to the number of **packet replicas** they generate. Which of them require a **preliminary route set-up phase**?

In the **decreasing order** of number of replicas generated:

Uncontrolled flooding :

The source node **sends a copy of the packet to all of its neighbors**. When a node receives a broadcast packet, it duplicates the packet and forwards it to all of its neighbors - except the neighbor from which it received the packet.

If the graph is connected, this scheme will eventually deliver a copy of the broadcast packet to all nodes in the graph. Although this scheme is simple and elegant, it has a fatal flaw: **if the graph has cycles**, then one or more copies of each broadcast packet will cycle **indefinitely**. When a node is connected to more than two other nodes, it will create and forward multiple copies of the broadcast packet, each of which will create multiple copies of itself (at other nodes with more than two neighbors), and so on. This broadcast storm, resulting from the endless multiplication of broadcast packets, would eventually result in so many broadcast packets being created that the network would be rendered useless.

Controlled flooding:

The key to avoiding a broadcast storm is for a node to judiciously choose **when to flood a packet** and (e.g., if it has already received and flooded an earlier copy of a packet) **when not to flood a packet**. 2 ways of controlled flooding:

Sequence-number-controlled flooding: a source node puts its **address** (or other **unique identifier**) as well as a **broadcast sequence number** into a broadcast packet, then sends the packet to all of its neighbors. Each node maintains a list of the **source address and sequence number of each broadcast packet** it has already received, duplicated, and forwarded. When a node receives a broadcast packet, it first checks whether the packet is in this list. If so, the packet is dropped; if not, the packet is duplicated and forwarded to all the node's neighbors - except the node from which the packet has just been received.

Reverse path forwarding (RPF): When a router receives a broadcast packet with a **given source address**, it transmits the packet on all of its **outgoing links** - except the one on which it was received only if the packet arrived on the link that is **on its own shortest unicast path back to the source**. Otherwise, the router simply discards the incoming packet.

Spanning-Tree Broadcast

While sequence-number-controlled flooding and RPF (controlled flooding) avoid broadcast storms, they do not completely avoid the **transmission of redundant broadcast packets**.

First construct a spanning tree. When a source node wants to send a broadcast packet, it sends the packet out on all of the incident links that **belong to the spanning tree**. A node receiving a broadcast packet then forwards the packet to all its neighbors in the spanning tree except the neighbor from which it received the packet. Not only does spanning tree eliminate redundant broadcast packets, but once in place, the spanning tree can be used by any node to begin a broadcast. The main complexity here is the **creation and maintenance** of the spanning tree.

RPF and Spanning-Tree Broadcast require a set-up phase: RPF needs the **shortest path info**, Spanning Tree obviously needs the **tree**